

Cross-browser testing

COMPLETE LEARNING GUIDE

What You'll Learn

Browsers differ because they're built on different rendering engines — Chromium-based browsers (Chrome, Edge, Brave, Opera) use Blink, Firefox uses Gecko, and Safari uses WebKit — each implementing CSS and JavaScript specifications on its own schedule, occasionally with real bugs or gaps in support for newer features. Cross-browser testing is the practice of verifying an application actually works correctly across this real diversity rather than only the browser a developer happens to use daily. This submodule covers feature detection versus browser sniffing, the tools that make gaps in support knowable ahead of time (caniuse.com, @supports), the mechanics of polyfills and CSS prefixing, and a practical testing strategy that combines fast automated checks with periodic real-device manual verification.

1. Why Browsers Render Differently

Each rendering engine (Blink, Gecko, WebKit) is a separate, independently developed codebase implementing the same web standards — differences arise both from genuinely differing implementation timelines for new CSS/JS features and from legitimate spec ambiguities interpreted slightly differently. Safari, running WebKit, has historically lagged behind Chromium on some newer CSS features (though this gap has narrowed significantly in recent years), and is the only rendering engine Apple permits on iOS regardless of which 'browser app' a user has installed — meaning every browser on iPhone (Chrome, Firefox, Edge included) is actually running WebKit underneath, which makes testing on an actual iOS device functionally equivalent to testing Safari specifically.

2. Feature Detection vs. Browser Sniffing

Browser sniffing — inspecting `navigator.userAgent` to guess which browser is running and branching logic accordingly — is fragile and actively discouraged: user agent strings are inconsistent, spoofable, and require constant maintenance as new browser versions/names appear. Feature detection checks directly whether the capability you need actually exists, which is both more reliable and self-updating as browsers evolve. In CSS, the `@supports` at-rule checks for property/value support directly in stylesheets; in JavaScript, checking for the existence of an API (`'IntersectionObserver'` in `window`) before using it accomplishes the same goal.

CSS

```
/* Feature detection in CSS – only applies grid layout if the browser supports it */
@supports (display: grid) {
  .gallery { display: grid; grid-template-columns: repeat(3, 1fr); }
}
@supports not (display: grid) {
  .gallery { display: flex; flex-wrap: wrap; } /* fallback for ancient browsers */
}
```

3. Checking Support Before You Build: caniuse.com

Before using any newer CSS property, JS API, or HTML element in production code, checking its current support on caniuse.com takes seconds and prevents shipping a feature that silently fails (or, worse, partially works with subtle bugs) for a meaningful share of real users. Caniuse reports support broken down by browser and version, along with known partial-support caveats and links to relevant bug trackers — it should be a standard, habitual check before adopting any feature you haven't used before, not something consulted only after a bug report comes in.

4. Polyfills and Autoprefixing

A polyfill is a piece of JavaScript that implements a missing browser API using older, more widely supported primitives, letting code call the modern API as if it were natively supported everywhere. CSS vendor prefixes (-webkit-, -moz-, -ms-) were the historical mechanism for browsers to ship experimental support for a not-yet-finalized CSS feature under a prefixed name — modern projects handle this automatically with Autoprefixer (usually run as part of the build via PostCSS), which reads a project's target browser list and adds exactly the prefixes still needed, rather than developers hand-writing every prefix.

CSS

```
/* You write plain, unprefixed CSS: */
.box {
  user-select: none;
}

/* Autoprefixer (build-time) outputs, based on your target browser list: */
.box {
  -webkit-user-select: none;
  -moz-user-select: none;
  user-select: none;
}
```

5. Automated Cross-Browser Testing

Modern end-to-end testing tools like Playwright can run the same test suite against Chromium, Firefox, and WebKit engines with a simple configuration change, catching rendering or behavior differences automatically as part of CI rather than relying purely on manual spot-checks. Cloud device-testing services (BrowserStack, Sauce Labs) extend this further by giving access to real device/OS/browser combinations that would otherwise be impractical to own and maintain physically, which matters especially for verifying older but

still commonly used browser versions.

JAVASCRIPT

```
// playwright.config.js – run the same suite across 3 real engines
module.exports = {
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
    { name: 'webkit', use: { browserName: 'webkit' } }, // Safari's engine
  ],
};
```

6. Testing Beyond Just 'Does It Render'

Cross-browser and cross-device testing should also cover user preference media features that some browsers/OSes support more completely than others: `prefers-color-scheme` (dark mode), `prefers-reduced-motion` (users who've disabled animations, often for vestibular or motion-sensitivity reasons), and `prefers-contrast`. A page that ignores `prefers-reduced-motion` and plays a large parallax animation regardless of the user's explicit OS-level preference is a cross-compatibility and accessibility failure, not merely a cosmetic one.

CSS

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

7. A Practical Testing Strategy

A realistic strategy layers three levels: automated cross-engine tests (Playwright/Cypress) running in CI on every change to catch regressions early and cheaply; periodic manual checks on real hardware for at least one Chromium browser, Firefox, and an actual iOS device (covering WebKit, since that's mandatory on iOS regardless of the browser app used) before major releases; and an ongoing habit of checking caniuse.com before adopting any unfamiliar modern feature, rather than discovering the gap only after a user reports a broken page.

Common Pitfalls

- Sniffing `navigator.userAgent` to detect a browser instead of checking directly whether the needed feature/API exists.
- Assuming 'testing in Chrome' covers Safari, when WebKit (Safari's engine) has historically diverged the most on both CSS and JS feature support.

- Using a brand-new CSS or JS feature in production without checking caniuse.com first, silently breaking the page for users on older browsers.
- Manually hand-writing vendor prefixes instead of using Autoprefixer, leading to inconsistent or outdated prefix coverage.
- Testing only on a resized desktop browser and never on a real iOS device, missing WebKit-specific bugs entirely.
- Ignoring prefers-reduced-motion and prefers-color-scheme, shipping an experience that overrides explicit user/OS accessibility preferences.

Key Takeaways

- Every browser on iOS runs WebKit underneath, regardless of which browser app is installed — real iOS device testing is effectively Safari testing.
- Feature detection (@supports, checking for an API's existence) is more reliable and future-proof than browser/user-agent sniffing.
- Check caniuse.com before adopting any unfamiliar modern CSS/JS feature — it takes seconds and prevents shipping silent breakage.
- Autoprefixer should handle vendor prefixes automatically as part of the build — don't hand-write them.
- A solid strategy layers automated cross-engine CI tests with periodic real-device manual verification, not one or the other alone.

Learn More

Can I use... — Browser support tables

<https://caniuse.com/>

MDN — Cross-browser testing

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Testing

Playwright — Cross-browser testing

<https://playwright.dev/docs/browsers>